

Kompilace kódu a verzování

Základy programování

Matouš Cejnek

U12110, FS, ČVUT v Praze

Co si dnes odnesete?

- Co je **program**, **algoritmus** a **programovací jazyk**?
- Jak se ze zdrojového kódu stane běžící program?
- Jak se liší **kompilace** a **interpretace**?
- K čemu slouží **IDE** a online běhové prostředí?
- Proč verzujeme a jak Git přemýšlí o historii projektu?

Co je algoritmus?

Algoritmus

Konečný a jednoznačný postup, který převádí vstup na požadovaný výstup.

- Popisuje **řešení problému**, ne konkrétní zápis v jazyce.
- Musí mít srozumitelné kroky a po konečném počtu kroků skončit.
- Tentýž algoritmus lze zapsat slovně, diagramem nebo programem.

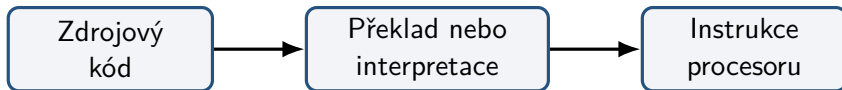
Co je program?

Program

Zápis algoritmu a souvisejících dat v podobě, kterou lze vykonat počítačem.

- **Programovací jazyk** určuje povolenou syntax a význam konstrukcí.
- **Zdrojový kód** je text vytvořený programátorem.
- Procesor přímo vykonává až **strojové instrukce**.

Proč počítač nespustí zdrojový kód přímo?



- Zdrojový jazyk je navržen především pro lidi.
- Procesor rozumí instrukční sadě konkrétní architektury.
- Mezi nimi musí existovat překladač nebo běhové prostředí.

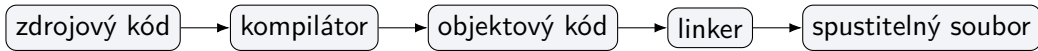
Co je kompilace?

Kompilace

Překlad zdrojového kódu do jiné reprezentace před jeho spuštěním.

- Kompilátor odhaluje část chyb ještě před spuštěním.
- Překlad může probíhat v několika navazujících krocích.
- Výsledný program může být závislý na operačním systému a procesoru.

Jak vypadá překladový řetězec?



- Každý nástroj řeší jinou část převodu programu.
- Konkrétní řetězec závisí na jazyce, překladači a platformě.

Co vytváří kompilátor?

Objektový kód

Přeložená podoba jedné části programu, která ještě nemusí být samostatně spustitelná.

- Obsahuje strojové instrukce a data pro cílovou platformu.
- Může stále odkazovat na funkce nebo data definovaná jinde.
- Větší projekt se obvykle překládá po samostatných částech.

Co dělá linker?

Linker neboli sestavovací program

Spojuje objektové soubory a knihovny do výsledného programu.

- Vyhledá, kde jsou definované používané funkce a proměnné.
- Propojí odkazy mezi jednotlivými částmi programu.
- Ohlásí chybu, pokud potřebná definice chybí nebo je nejednoznačná.

Co je knihovna?

Knihovna

Znovupoužitelný soubor již připraveného kódu, který program využívá.

- **Statická knihovna:** potřebné části se vloží do programu při linkování.
- **Dynamická knihovna:** načte se při spuštění nebo až během běhu.
- Knihovna není samostatná aplikace; poskytuje funkce jiným programům.

Co se děje při spuštění?

- Operační systém načte spustitelný soubor do paměti.
- **Loader** připraví program a potřebné dynamické knihovny.
- Runtime inicializuje prostředí, které program potřebuje.
- Řízení přejde na vstupní bod programu.

Spustitelný neznamená přenositelný

Soubor vytvořený pro jednu platformu nemusí fungovat na jiné.

Co je interpretace?

Interpretace

Interpret načítá program a zajišťuje vykonávání jeho instrukcí za běhu.

- Ke spuštění potřebujeme zdrojový kód a vhodný interpret/runtime.
- Chyba se může projevit až při vykonání problematické části programu.
- Typický příklad uživatelského pohledu: `python program.py`.

Pozor

„Interpretovaný jazyk“ neznamená, že uvnitř neprobíhá žádný překlad.

Interpret a virtuální stroj

- **Interpret** zajišťuje vykonávání programu za běhu.
- **Virtuální stroj** vykonává instrukce mezireprezentace, například bytecode.
- Stejná mezireprezentace může běžet na více platformách s vhodným runtime.

Názvy popisují vrstvy, ne vždy celý jazyk

Jeden jazyk může mít více implementací s odlišným způsobem vykonávání.

Kompilace, nebo interpretace?

Překlad před spuštěním

- vzniká cílový kód,
- více práce předem,
- často rychlý běh.

Překlad během běhu

- nutné běhové prostředí,
- snadná přenositelnost,
- pružná práce s programem.

V praxi jde o spektrum

Jazyky a implementace mohou kombinovat kompilaci, bytecode, virtuální stroj i JIT překlad.

Kde je v tomto obrazu Python?

- Python je **programovací jazyk**; CPython je jeho běžná implementace.
- CPython typicky překládá zdrojový kód do **bytecode**.
- Bytecode vykonává virtuální stroj uvnitř Python runtime.



Co je runtime?

Běhové prostředí (runtime)

Služby a pravidla potřebná při spuštění programu.

- načtení programu a knihoven,
- správa paměti a objektů,
- vstup, výstup a komunikace s operačním systémem,
- zpracování běhových chyb.

Jazyk je specifikace; **implementace** a runtime ji uskutečňují.

Chyby před spuštěním

- **Syntaktická chyba:** zápis neodpovídá pravidlům jazyka.
- **Překladová chyba:** program nelze převést do cílové podoby.
- **Linkovací chyba:** některou požadovanou definici nelze najít nebo propojit.

Výhoda včasné kontroly

Část problémů lze najít ještě dříve, než program dostane vstup a začne pracovat.

Chyby během a po spuštění

- **Běhová chyba:** problém vznikne až během vykonávání.
- **Logická chyba:** program běží, ale dává nesprávný výsledek.

Důležitý rozdíl

Úspěšný překlad ani spuštění nedokazují, že program řeší správný problém.

Co je IDE?

IDE — integrované vývojové prostředí

Aplikace, která spojuje nástroje pro tvorbu a zkoumání programu.

- editor kódu a navigace v projektu,
- zvýraznění syntaxe a průběžná analýza,
- spuštění, sestavení a práce s runtime,
- debugger, testy, terminál a integrace verzování.

Textový editor, nebo IDE?

- **Textový editor**: především úprava souborů; další nástroje doplňujeme.
- **IDE**: ucelené prostředí s přednastaveným workflow projektu.

Volba nástroje

Nástroj mění způsob práce, nikoli význam programu ani pravidla jazyka.

Notebook a cloudové prostředí

- **Notebook**: dokument kombinující text, kód a jednotlivé výstupy.
- **Cloudové IDE / Colab**: běh na vzdáleném počítači přes prohlížeč.
- Výhodou je rychlý začátek; omezením může být závislost na službě a připojení.

Proč verzovat?

- Víme, **co**, **kdo** a **proč** změnil.
- Můžeme porovnat stavy projektu a vrátit se ke starší verzi.
- Více lidí může bezpečně rozvíjet tentýž projekt.
- Experiment lze oddělit od stabilní práce.

Složky *final*, *final2*, *opravdu-final*

Kopie souborů nejsou spolehlivá historie: chybí vztahy, autor, důvod změny i možnost rozumného sloučení.

Co je Git?

Git

Distribuovaný systém správy verzí, který uchovává historii projektu jako propojené snímky.

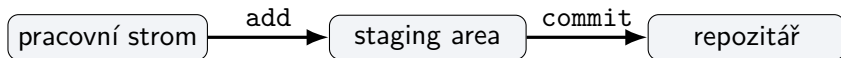
- **Repozitář** obsahuje soubory projektu i jejich historii.
- Každý klon má obvykle úplnou lokální historii.
- Většina operací funguje lokálně, bez připojení k serveru.
- Git není totéž co GitHub: Git je nástroj, GitHub je hostingová služba.

Git ukládá snímky, ne názvy verzí

- Každý commit představuje snímek stavu sledovaných souborů.
- Nezměněný obsah Git nemusí ukládat znovu; odkáže na existující data.
- Commity jsou propojené a vytvářejí historii projektu.

A -- B -- C -- D

Jak Git vidí změnu?



- **Modified**: soubor je změněný v pracovním stromu.
- **Staged**: konkrétní podoba změny je vybrána pro další commit.
- **Committed**: snímek je bezpečně uložený v lokální historii.

Co je commit?

Commit

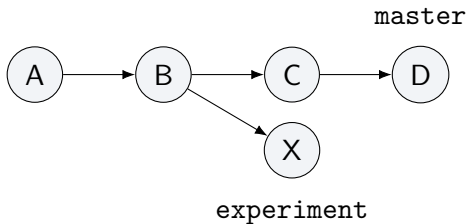
Pojmenovaný snímek projektu s autorem, časem, zprávou a odkazem na předchozí historii.

- Zachycuje jeden smysluplný krok vývoje.
- Identifikuje jej hash odvozený z jeho obsahu a metadat.
- Dobrá zpráva vysvětluje záměr změny, nejen jména souborů.

A -- B -- C ← historie snímků

Co je větev?

Větev (branch) je pohyblivý ukazatel na commit; umožňuje rozvíjet více linií historie.



- Vytvoření větve nekopíruje celý projekt.
- **Merge** znovu propojí dvě linie vývoje.

Co je merge a konflikt?

- **Merge** spojí změny ze dvou linií historie.
- Nezávislé změny Git často sloučí automaticky.
- **Konflikt** vznikne, když nelze bezpečně rozhodnout, kterou podobu zachovat.
- Konflikt není porucha Gitu; je to rozhodnutí, které musí udělat člověk.

Lokální a vzdálený repozitář

- **Lokální repozitář:** pracovní kopie a historie na našem počítači.
- **Vzdálený repozitář:** další kopie historie, často na GitHubu.
- `clone`: vytvoří lokální kopii vzdáleného repozitáře.
- `push`: zveřejní lokální commity; `pull`: převezme vzdálené změny.

Častý omyl

Commit ukládá změnu lokálně. Teprve `push` ji odešle na vzdálený server.

Fork není clone

Fork

Kopie cizího repozitáře pod naším účtem na GitHubu.

- vzniká na serveru,
- můžeme do ní zapisovat,
- zůstává propojená s původním projektem.

Pořadí pro cvičení

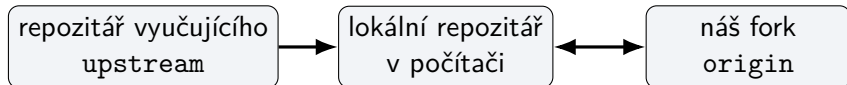
Nejdřív vytvoříme fork na GitHubu, potom naklonujeme svůj fork.

Clone

Lokální kopie vzdáleného repozitáře v našem počítači.

- vzniká příkazem `git clone`,
- obsahuje soubory i historii,
- umožňuje lokálně upravovat a commitovat.

Proč potřebujeme origin a upstream?



- **Remote** je pojmenovaný odkaz na vzdálený repozitář.
- `origin` ukazuje na náš fork; `upstream` na původní repozitář.
- Z `upstream` změny načítáme; do `origin` je odesíláme.
- Adresy vždy ověříme pomocí `git remote -v`.

Jak synchronizovat master?

- 1 Přepneme se na lokální hlavní větev: `git switch master`.
- 2 Načteme stav původního projektu: `git fetch upstream`.
- 3 Začleníme změny: `git merge upstream/master`.
- 4 Aktualizujeme svůj fork: `git push origin master`.

`fetch` pouze načte vzdálenou historii; teprve `merge` změní naši aktuální větev.

Před založením pracovní větve

Novou větev vytváříme z aktualizovaného `master`, jinak začínáme pracovat na zastaralém základu.

Co je pull request?

Pull request

Návrh na začlenění jedné větve do druhé.

- Ukazuje rozdíl mezi zdrojovou a cílovou větví.
- Umožňuje změny před sloučením zkontrolovat a okomentovat.
- Na GitHubu může spojovat větve ve stejném i v různých repozitářích.

Pozor na cílový repozitář

V tomto cvičení cílíme na svůj fork, ne na repozitář vyučujícího.

Kam poslat změny?

- 1 Z aktualizovaného master vytvoříme pracovní větev.
- 2 Změny commitujeme lokálně do pracovní větve.
- 3 Větev odešleme pomocí `git push` do svého origin.
- 4 Na GitHubu vytvoříme pull request do master svého forku.

Před potvrzením: Zkontrolujeme zdrojovou i cílovou větev a oba repozitáře.

Jeden projekt, dvě vrstvy nástrojů

Vznik a běh programu

- zdrojový kód,
- překladač / interpret,
- runtime,
- IDE.

Historie projektu

- pracovní strom,
- staging area,
- commity a větve,
- vzdálený repozitář.

IDE může obě vrstvy zpřístupnit, ale nenahrazuje jejich pochopení.

Co si zapamatovat?

- Zdrojový kód musí být přeložen nebo vykonán prostřednictvím runtime.
- Kompilace a interpretace nejsou dvě dokonale oddělené kategorie.
- IDE spojuje nástroje, ale programovací jazyk ani Git se na něj nevážou.
- Git ukládá propojené snímky projektu v commitech.
- Fork je naše serverová kopie; clone je lokální kopie v počítači.
- `origin` míří na náš fork, `upstream` na původní projekt.

Otázky?